

Hadoop and MapReduce Setup - Step-by-Step Breakdown

1. Hadoop Services Management (Start HDFS and YARN)

Start HDFS (Distributed File System):

Command: `start-dfs.sh`

Output: All services started successfully.

Start YARN (Yet Another Resource Negotiator):

Command: `start-yarn.sh`

Output: All services started successfully.

Check Running Hadoop Services:

Command: `jps`

Output:

- NameNode
- DataNode
- ResourceManager
- NodeManager
- SecondaryNameNode
- Jps

2. MapReduce Setup Using Python Scripts

Write Mapper and Reducer Scripts:

Commands:

`nano mapper.py`

`nano reducer.py`

Make Scripts Executable:

Commands:

`chmod +x mapper.py`

`chmod +x reducer.py`

3. Testing MapReduce Locally (Without Hadoop Streaming Yet)

Create a Sample Input File:

Command:

```
echo "hello world welcome to the world of big data hello" > input.txt
```

Run MapReduce Pipeline Manually:

Command:

```
cat input.txt | ./mapper.py | sort | ./reducer.py
```

Expected Output:

```
big 1
data 1
hello 2
of 1
the 1
to 1
welcome 1
world 2
```

Explanation:

1. mapper.py -> Outputs key-value pairs like (word, 1)
2. sort -> Groups same words together
3. reducer.py -> Sums counts for each word

4. Mapper Code (mapper.py)

```
#!/usr/bin/env python3
```

```
import sys
```

```
for line in sys.stdin:
```

```
    line = line.strip()
```

```
    words = line.split()
```

```
    for word in words:
```

```
        print(f'{word}\t1')
```

5. Reducer Code (reducer.py)

```
#!/usr/bin/env python3
```

```
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split("\t")
    try:
        count = int(count)
    except ValueError:
        continue

    if current_word == word:
        current_count += count
    else:
        if current_word:
            print(f'{current_word}\t{current_count}')
        current_word = word
        current_count = count

if current_word == word:
    print(f'{current_word}\t{current_count}')
```